LANGGRAPH

Complete Developer Study Notes

Concepts • Practical Examples • Projects • Interview Preparation

Prepared by: Rahul Yadav

20 Pages

Rahul Yadav

1. Introduction to LangGraph

1.1 What is LangGraph ?

- LangGraph is a framework from LangChain used to build stateful, multi-agent and graph-based applications.
- It allows us to define workflows as a graph where each node performs a specific task and edges control the flow between nodes.
- LangGraph is useful for building complex applications like agentic workflows, chatbots, RAG systems, and multi-step automation.

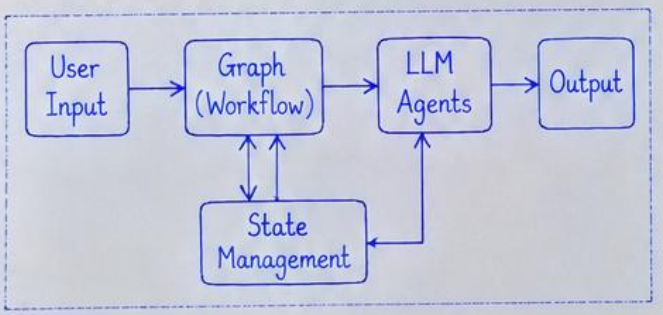
1.2 Why LangGraph ?

- Traditional LLM applications are linear and stateless, which are limited for complex use cases.
- LangGraph provides state management, conditional routing, loops and human-in-the-loop support.
- It helps in creating more reliable, controllable and scalable LLM applications.

1.3 Key Features

- Graph-based workflow design
- State management across steps
- Conditional routing and branching
- Support for loops and iterative workflows
- Multi-agent systems
- Human-in-the-loop integration
- Integration with LangChain tools, LLMs and external services

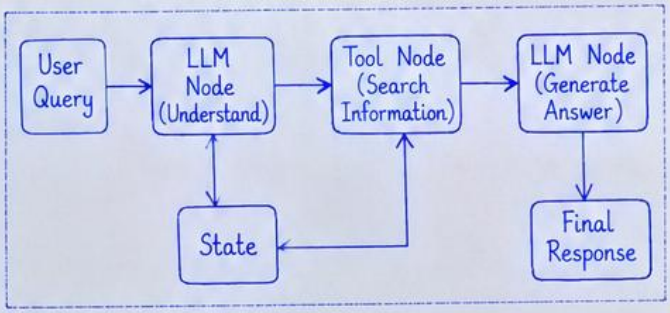
1.4 Basic Architecture



1.5 Core Components

- State : Shared data that flows between nodes.
- Node : A function or agent that performs a task.
- Edge : Defines the transition from one node to another.
- Graph : Collection of nodes and edges that define the workflow.
- Tool : External function or API used by nodes.
- LLM : Language model used for reasoning and generating responses.

1.6 Simple Example Workflow



1.7 Use Cases

- RAG-based Q&A systems
- Multi-agent research and analysis
- Customer support automation
- Document processing workflows
- Data analysis and reporting
- Agentic AI applications
- Human-in-the-loop (approval, review, feedback)
- Workflow automation with tools and APIs

1.8 LangGraph vs LangChain Chains

Feature	LangChain Chains	LangGraph
Workflow Type	Linear	Graph-based
State Management	Limited	Built-in
Conditional Routing	Difficult	Easy
Loops / Iterations	Not supported	Supported
Multi-Agent Support	Limited	Native support
Complex Workflows	Hard to manage	Easier and scalable

1.9 Advantages

- Better control over workflow execution.
- Supports complex, real-world use cases.
- Stateful and reliable applications.
- Easy integration with LangChain ecosystem.
- Flexible and extensible design.

1.10 Limitations

- Slightly steeper learning curve.
- More configuration compared to simple chains.
- Need to manage state and graph design carefully.
- Not always required for simple use cases.
- Performance depends on proper graph design.

1.11 Summary

- LangGraph helps us build stateful and graph-based LLM applications.
- It is more powerful than traditional chains.
- It is widely used for agentic systems, RAG and complex automation workflows.

Rahul Yadav

2. Core Concepts and Terminology

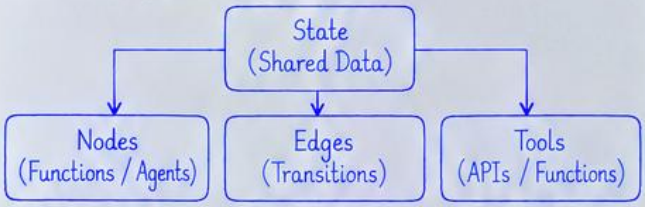
2.1 Overview

- LangGraph is built on top of LangChain and provides a graph-based way to build stateful, multi-agent and long-running applications.
- It uses nodes, edges and a shared state to control the flow of execution.
- Understanding core concepts is important to design correct and scalable workflows.

2.2 Key Terminology

- Graph : A collection of nodes and edges.
- Node : A function or agent that performs a specific task.
- Edge : Defines the transition from one node to another.
- State : Shared data that flows between nodes.
- Message : A single piece of information in the conversation (e.g., user, assistant or tool message).
- Conditional Edge : Route to different nodes based on a condition.
- Entry Point : Starting node of the graph.
- End Point : Final node where the workflow ends.
- Tool : External function or API used by a node.
- Checkpoint : Saves the state for persistence and resuming the workflow.

2.3 Graph Components



- Process input
- Update state
- Call tools
- Return output

- Control flow
- Conditional routing
- Define next node

- External APIs
- Search, calculator
- Database access
- Custom functions

2.4 State in LangGraph

- State is a shared dictionary-like object that holds data across the graph.
- Each node can read and update the state.
- Common fields in state : messages, user input, intermediate results, tool outputs, etc.

Example State Structure:

```
state = {
    "messages": [
        {"role": "user", "content": "What is LangGraph?"}
    ],
    "user_input": "What is LangGraph?",
    "intermediate_data": {},
    "final answer": ""
}
```

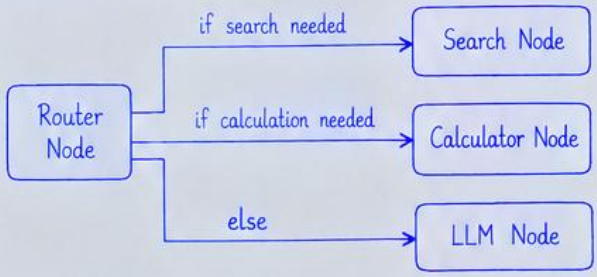
2.5 Types of Nodes

- LLM Node : Uses an LLM to generate a response.
- Tool Node : Calls an external tool or API.
- Agent Node : Uses an agent with tools and reasoning.
- Custom Node : A user-defined function.
- Router Node : Decides next step based on condition.

2.6 Types of Edges

- Normal Edge : Always goes to the next node.
- Conditional Edge : Routes based on a condition (e.g., LLM output, tool result, or state).
- Loop Edge : Allows repeating a set of nodes until a condition is met.
- End Edge : Terminates the workflow.

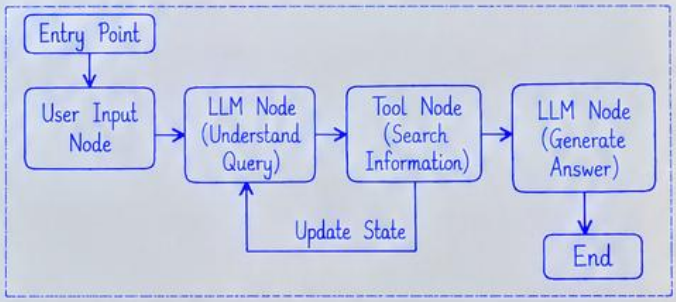
Conditional Edge Example:



2.7 Types of Graphs

- Sequential Graph : Nodes execute in a fixed order.
- Conditional Graph : Uses conditions to route flow.
- Loop Graph : Repeats nodes until a stopping condition.
- Multi-Agent Graph : Multiple agents working together.
- Hierarchical Graph : Complex workflows with subgraphs.

2.8 Simple Graph Example



2.9 Important Concepts

- State Management : Keeps the conversation context and intermediate results.
- Workflow : Defines how nodes are connected and executed.
- Routing : Controls the path of execution using conditional edges.
- Parallel Execution : Multiple nodes can run in parallel (in advanced use cases).
- Human-in-the-Loop : Add human approval or feedback in the workflow.
- Persistence : Save and resume the state using check-pointing (e.g., memory, database).
- Observability : Track execution using LangSmith or custom logging.

Rahul Yadav

3. Setting up Development Environment

3.1 Prerequisites

- Python 3.9 or higher
- Basic understanding of Python programming
- Familiarity with LLM concepts (OpenAI, etc.)
- Install a code editor (VS Code / PyCharm)
- API keys for LLM providers (e.g., OpenAI)
- Basic knowledge of virtual environments

3.2 Create a Project Setup

1. Create a new project directory.
2. Create and activate a virtual environment.
3. Install required dependencies.
4. Set up API keys and environment variables.
5. Create a basic folder structure.

3.3 Folder Structure

```
langgraph-project/          # Project root
├── app/                    # Application code
│   ├── main.py             # Main file to run
│   ├── graph.py            # Graph definition
│   └── nodes.py            # Node functions
│
├── .env                    # Environment variables
├── requirements.txt        # Dependencies
└── README.md               # Project documentation
```

3.4 Install Dependencies

```
# Create virtual environment
python -m venv venv
# Activate (Windows)
venv\Scripts\activate
# Activate (Mac/Linux)
source venv/bin/activate

# Install required packages
pip install langgraph langchain openai python-dotenv
pip install langchain-community
pip install tavily-python # (optional for web search)
pip install streamlit     # (optional for UI)
```

3.5 Set up Environment Variables

```
# .env file
OPENAI_API_KEY=your_openai_api_key
TAVILY_API_KEY=your_tavily_api_key
LANGCHAIN_TRACING_V2=true
LANGCHAIN_API_KEY=your_langsmith_api_key
LANGCHAIN_PROJECT=langgraph-project
```

- Use python-dotenv to load the variables.
- Never hardcode API keys in the code.

3.6 Basic Setup and Test

```
from dotenv import load_dotenv
from langchain_openai import ChatOpenAI
import os

load_dotenv()

llm = ChatOpenAI(
      model="gpt-3.5-turbo",
      temperature=0.0,
      api_key=os.getenv("OPENAI_API_KEY")
)

response = llm.invoke("Hello! Are you working?")
print(response.content)
```

- This verifies that your environment is correctly set up and the LLM is working.

3.7 Install and Verify LangGraph

```
import langgraph
print(langgraph.__version__)

from langgraph.graph import StateGraph, END

print("LangGraph is installed and ready!")
```

- Make sure you are using the latest stable version.
- Refer to official documentation for version updates.

3.8 Simple Hello World Graph

```
from typing import TypedDict
from langgraph.graph import StateGraph, END

class State(TypedDict):
      message: str

def my_node(state: State):
      return {"message": "Hello from LangGraph!"}

graph = StateGraph(State)
graph.add_node("hello", my_node)
graph.set_entry_point("hello")
graph.set_finish_point("hello")

app = graph.compile()

result = app.invoke({"message": "Hi"})
print(result)
```

3.9 Next Steps

- Explore different types of nodes (LLM, tool, custom).
- Learn about state management and routing.
- Integrate external tools and APIs.
- Build more complex multi-step workflows.
- Move to real-world examples in next sections.

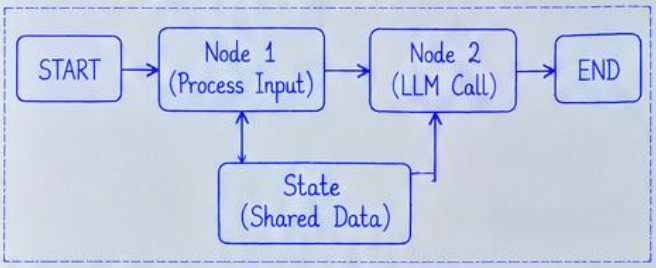
Rahul Yadav

4. Building a Basic Graph

4.1 What is a Graph in LangGraph ?

- A graph is a collection of nodes and edges that define a workflow.
- Each node performs a specific task (e.g., LLM, tool or custom function).
- Edges define how the flow moves from one node to another.
- We can build simple or complex graphs depending on the use case.

4.2 Basic Graph Structure



4.3 Define a Simple Node

- A node is a Python function that takes the current state as input and returns an updated state.
- Nodes can perform tasks like data preprocessing, LLM calls, tool execution, etc.

```
from typing import TypedDict
class State(TypedDict):
    message: str
    result: str

def process_input(state: State) -> State:
    user_msg = state["message"]
    state["result"] = f"Processed: {user_msg}"
    return state
```

4.4 Another Node with LLM

- This node will call an LLM (e.g., OpenAI) and generate a response.
- We can use ChatOpenAI from langchain-openai.

```
from langchain_openai import ChatOpenAI

llm = ChatOpenAI(model="gpt-3.5-turbo",
                 temperature=0.0)

def call_llm(state: State) -> State:
    user_msg = state["result"]
    response = llm.invoke(user_msg)
    state["result"] = response.content
    return state
```

4.5 Initialize the Graph

- We use StateGraph from langgraph.graph.
- Add nodes, edges and define entry and finish points.
- Finally, compile the graph to get a runnable app.

4.6 Complete Code Example

```
from langgraph.graph import StateGraph, END
# Define State
class State(TypedDict):
    message: str
    result: str

# Define Nodes
def process_input(state: State) -> State:
    user_msg = state["message"]
    state["result"] = f"Processed: {user_msg}"
    return state

from langchain_openai import ChatOpenAI
llm = ChatOpenAI(model="gpt-3.5-turbo", temperature=0.0)

def call_llm(state: State) -> State:
    response = llm.invoke(state["result"])
    state["result"] = response.content
    return state

# Build Graph
graph = StateGraph(State)
graph.add_node("process", process_input)
graph.add_node("llm", call_llm)

graph.set_entry_point("process")
graph.add_edge("process", "llm")
graph.add_edge("llm", END)

app = graph.compile()
```

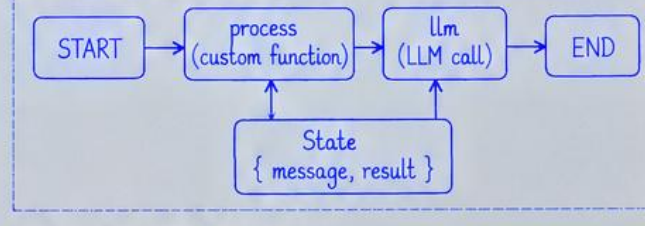
4.7 Run the Graph

```
input_state = {"message": "What is LangGraph?"}
result = app.invoke(input_state)
print(result)
```

4.8 Expected Output (Example)

```
{
  'message': 'What is LangGraph?',
  'result': 'LangGraph is a framework from LangChain
      used to build stateful, multi-agent and graph-based
      applications.'
}
```

4.9 Visual Representation



4.10 Key Takeaways

- A graph is built using StateGraph.
- Each node is a Python function.
- Edges define the flow between nodes.
- The graph is compiled and then invoked.
- This is the simplest form of a LangGraph application and can be extended with tools, conditional routing, and multi-agent workflows.

Rahul Yadav

5. State Management in LangGraph

5.1 What is State in LangGraph?

- State is a shared data object that flows between nodes in the graph.
- It stores the conversation context, intermediate results and other data required during workflow execution.
- Each node can read from and update the state.
- State helps in building multi-step, conditional and agentic workflows.

5.2 Key Characteristics

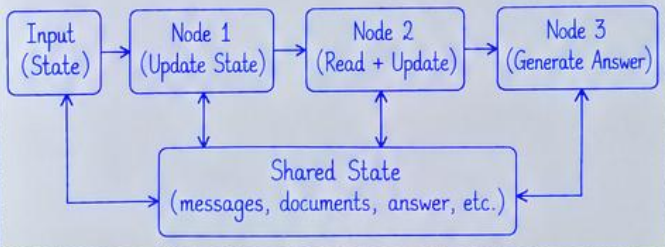
- Shared across all nodes
- Can be updated by any node
- Supports custom data structures (TypedDict / Pydantic)
- Automatically passed between nodes
- Useful for maintaining conversation history, tool outputs, references, etc.

5.3 State Structure (TypedDict Example)

```
from typing import TypedDict, List, Annotated

class GraphState(TypedDict):
    messages: Annotated[List[dict], "Conversation history"]
    query: str                  # User input
    documents: List[str]        # Retrieved documents
    answer: str                 # Final answer
    steps: int                  # Track number of steps
```

5.4 State Flow in a Graph



5.5 Reading and Updating State in a Node

```
def retrieve_node(state: GraphState) -> GraphState:
    query = state["query"]
    docs = ["Doc1 about " + query, "Doc2 about " + query]
    state["documents"] = docs
    state["steps"] = state.get("steps", 0) + 1
    return state
```

```
def generate_node(state: GraphState) -> GraphState:
    docs = state["documents"]
    answer = f"Answer based on: {docs}"
    state["answer"] = answer
    state["steps"] = state.get("steps", 0) + 1
    return state
```

5.6 Complete Example with State Management

```
from langgraph.graph import StateGraph, END
from typing import TypedDict, List, Annotated

class GraphState(TypedDict):
    messages: Annotated[List[dict], "Conversation history"]
    query: str
    documents: List[str]
    answer: str
    steps: int

def retrieve(state: GraphState) -> GraphState:
    query = state["query"]
    state["documents"] = ["Doc1", "Doc2"]
    state["steps"] = state.get("steps", 0) + 1
    return state

def generate(state: GraphState) -> GraphState:
    docs = state["documents"]
    state["answer"] = f"Final answer based on {docs}"
    state["steps"] = state.get("steps", 0) + 1
    return state

# Build graph
graph = StateGraph(GraphState)
graph.add_node("retrieve", retrieve)
graph.add_node("generate", generate)
graph.set_entry_point("retrieve")
graph.add_edge("retrieve", "generate")
graph.add_edge("generate", END)

app = graph.compile()

# Run graph
initial_state = {"query": "What is LangGraph?", "steps": 0}
result = app.invoke(initial_state)
print(result)
```

5.7 Expected Output

```
{
  'messages': [],
  'query': 'What is LangGraph?',
  'documents': ['Doc1', 'Doc2'],
  'answer': 'Final answer based on [\'Doc1\', \'Doc2\']',
  'steps': 2
}
```

5.8 Important Points

- State must be a TypedDict or Pydantic model.
- Each node must return the updated state.
- Use meaningful field names for better readability.
- Keep state structure simple and modular.
- You can store lists, dictionaries, strings, numbers, etc.

5.9 Summary

- State is the core of LangGraph workflows.
- It allows nodes to share and update data.
- Proper state design helps in building complex multi-agent and graph-based applications.
- Always validate and manage state carefully.

Rahul Yadav

6. LLM Integration with LangGraph

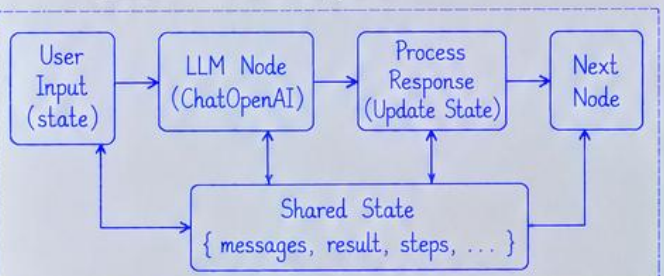
6.1 Overview

- LangGraph allows us to integrate LLMs (e.g., OpenAI) into graph workflows.
- LLM nodes can read the current state, generate responses and update the state.
- We can combine LLMs with tools, conditional routing and multi-step logic.
- This enables building advanced applications like chatbots, research assistants, document analysis and agentic systems.

6.2 How LLM Integration Works ?

- We create a node that calls an LLM (e.g., ChatOpenAI).
- The node takes input from the state (e.g., user message).
- We define a prompt using the input state.
- The LLM generates a response.
- The response is stored back in the state.
- We can route to next nodes based on the LLM output.

6.3 Architecture Diagram



6.4 Set up OpenAI LLM

```
from langchain_openai import ChatOpenAI
import os
# Set your API key (use environment variable)
os.environ["OPENAI_API_KEY"] = "your_openai_api_key"

# Initialize LLM
llm = ChatOpenAI(
      model="gpt-3.5-turbo",
      temperature=0.0
)
```

6.5 LLM Node Example

- This node takes the user message from state.
- It calls the LLM with a prompt.
- The LLM response is stored back in state.
- We can also maintain conversation history.

```
def llm_node(state: dict) -> dict:
    messages = state["messages"]

    response = llm.invoke(messages)

    # Update state
    state["messages"].append(response)
    state["result"] = response.content
    return state
```

6.6 Full Code Example (Simple Q&A Graph)

```
from langgraph.graph import StateGraph, END
from typing import TypedDict, List, Annotated
from langchain_openai import ChatOpenAI
import os

class GraphState(TypedDict):
    messages: Annotated[List[dict], "Conversation history"]
    result: str
    steps: int

# Initialize LLM
os.environ["OPENAI_API_KEY"] = "your_openai_api_key"
llm = ChatOpenAI(model="gpt-3.5-turbo", temperature=0.0)

# Define LLM node
def ask_llm(state: GraphState) -> GraphState:
    messages = state["messages"]
    response = llm.invoke(messages)
    state["messages"].append(response)
    state["result"] = response.content
    state["steps"] = state.get("steps", 0) + 1
    return state

# Build graph
graph = StateGraph(GraphState)
graph.add_node("ask_llm", ask_llm)
graph.set_entry_point("ask_llm")
graph.add_edge("ask_llm", END)

# Compile
app = graph.compile()

# Run
initial_state = {"messages": [{"role": "user", "content":
  "What is LangGraph?"}], "steps": 0}
result = app.invoke(initial_state)
print(result)
```

6.7 Expected Output (Example)

```
{
 'messages': [
    {'role': 'user', 'content': 'What is LangGraph?'},
    {'role': 'assistant', 'content': 'LangGraph is a framework
     built on top of LangChain to create stateful, multi-agent
     and graph-based applications. ...'},
 ],
 'result': 'LangGraph is a framework built on top of LangChain
    to create stateful, multi-agent and graph-based applications. ...',
    'steps': 1
}
```

6.8 Key Points

- Use ChatOpenAI or any other LLM provider.
- Maintain conversation history in the state.
- You can use system prompts, custom prompts and templates.
- Combine LLM nodes with tool nodes and conditional routing.
- Handle errors and retries for production use.

6.9 Summary

- LangGraph makes it easy to integrate LLMs into workflows.
- LLM nodes read from state, generate responses and update state.
- This enables building powerful applications like chatbots, RAG systems and multi-agent workflows.

Rahul Yadav

7. Tools and External Integrations

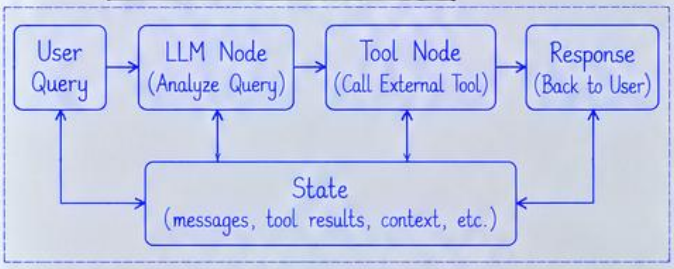
7.1 Overview

- Tools allow LangGraph to interact with external systems like web search, databases, APIs, calculators, files, etc.
- LLM can decide when to use a tool based on the user query and context.
- We can integrate both built-in tools and custom tools.
- Tools make the application more powerful and real-world ready.

7.2 Types of Tools

- Built-in Tools : Search, calculator, web browsing, etc.
- API Tools : Call external REST APIs.
- Database Tools : Query databases (e.g., PostgreSQL, MySQL).
- File Tools : Read and write files.
- Custom Tools : User-defined functions for specific tasks.
- Third-party Integrations : SerpAPI, Tavily, Wikipedia, Youtube, Slack, etc.

7.3 Architecture with Tools



7.4 Built-in Tools Example (Tavily Search)

- Tavily is a search tool that provides real-time web search results.
- We can integrate it using LangChain tools.

```
from langchain_community.tools import TavilySearchResults

# Initialize tool
search_tool = TavilySearchResults(
    api_key="your_tavily_api_key",
    max_results=3
)
```

7.5 Create a Tool Node

- We define a node that uses the tool to search information.
- The result is stored back in the state.

```
def search_node(state: dict) -> dict:
    query = state["query"]
    result = search_tool.invoke(query)
    state["search_results"] = result
    return state
```

7.6 Custom Tool Example (Calculator)

- We can create our own tools for specific use cases.
- This example performs a simple calculation.

```
from langchain.tools import tool
@tool
def calculator(expression: str) -> str:
    """Calculate a mathematical expression."""
    try:
        return str(eval(expression))
    except Exception as e:
        return f"Error: {str(e)}"
```

7.7 Complete Example (Search + Calculator)

```
from langgraph.graph import StateGraph, END
from typing import TypedDict, List, Annotated
from langchain_community.tools import TavilySearchResults
from langchain.tools import tool

# Define state
class GraphState(TypedDict):
    messages: Annotated[List[dict], "Conversation history"]
    query: str
    search_results: str
    calculation_result: str
    final_answer: str

# Initialize tools
search_tool = TavilySearchResults(api_key="your_tavily_api_key",
                                  max_results=3)

@tool
def calculator(expression: str) -> str:
    try:
        return str(eval(expression))
    except Exception as e:
        return f"Error: {str(e)}"

# Nodes
def search_node(state: GraphState) -> GraphState:
    query = state["query"]
    result = search_tool.invoke(query)
    state["search_results"] = str(result)
    return state

def calc_node(state: GraphState) -> GraphState:
    expr = state["query"]
    result = calculator.invoke(expr)
    state["calculation_result"] = result
    return state

def answer_node(state: GraphState) -> GraphState:
    search = state.get("search_results", "")
    calc = state.get("calculation_result", "")
    state["final_answer"] = f"Search: {search}\nCalculation: {calc}"
    return state

# Build graph
graph = StateGraph(GraphState)
graph.add_node("search", search_node)
graph.add_node("calculator", calc_node)
graph.add_node("answer", answer_node)
graph.set_entry_point("search")
graph.add_edge("search", "calculator")
graph.add_edge("calculator", "answer")
graph.add_edge("answer", END)

app = graph.compile()

# Run
initial_state = {"query": "What is LangGraph and 10*5?", "messages":[]}
result = app.invoke(initial_state)
print(result["final_answer"])
```

7.8 Expected Output (Example)

```
Search: LangGraph is a framework on top of LangChain...
Calculation: 50
```

7.9 Key Points

- Tools extend the capability of LLMs.
- We can use built-in, API, database or custom tools.
- LLM decides when to call a tool based on the query.
- Tool results are stored in the state for further processing.
- Always handle errors and retries while calling external tools.
- Use proper API keys and rate limiting for production.

Rahul Yadav

8. Conditional Routing and Branching

8.1 Overview

- Conditional routing allows the graph to decide the next node based on the state, LLM output or custom logic.
- It helps in building dynamic and flexible workflows.
- We can branch to different nodes based on conditions (e.g., user intent, tool result, category, etc.).
- It is useful for multi-step, multi-path and agentic applications.

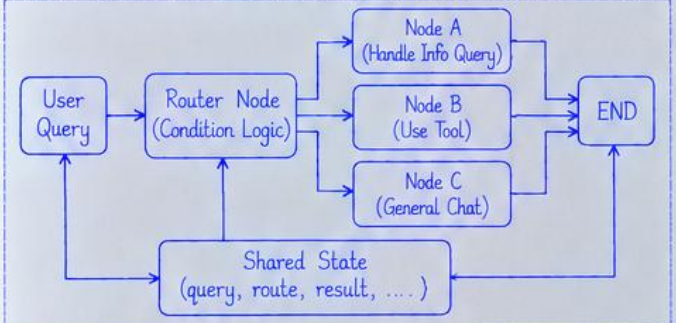
8.2 When to Use Conditional Routing ?

- Different actions based on user query type.
- Routing to different tools (search, calculator, etc.).
- Handling success / failure scenarios.
- Creating multi-agent workflows.
- Implementing loops and iterative processes.

8.3 Types of Conditional Routing

1. Simple Condition - If-else based routing.
2. LLM Based Routing - Use LLM to decide next node.
3. Tool Based Routing - Route based on tool output.
4. Multi-path Routing - Multiple possible next nodes.

8.4 Architecture Diagram



8.5 Define State

```
from typing import TypedDict, List, Annotated

class GraphState(TypedDict):
    messages: Annotated[List[dict], "Conversation history"]
    query_type: str      # Type of user query
    result: str          # Final response
    steps: int           # Track steps
```

8.6 Define Routing Logic

- We create a router function that analyzes the input and returns the next node.
- It can be based on keywords, LLM classification or rule-based logic.

```
def route_query(state: GraphState) -> str:
    query = state["messages"][-1]["content"].lower()
    if "calculate" in query or "math" in query:
        return "calculator"
    elif "search" in query or "latest" in query:
        return "search"
    else:
        return "general_chat"
```

8.7 Define Nodes

- Each node performs a specific task.
- Nodes update the state and return it.

```
from langchain_openai import ChatOpenAI
from langchain_community.tools import TavilySearchResults

llm = ChatOpenAI(model="gpt-3.5-turbo", temperature=0.0)
search_tool = TavilySearchResults(api_key="your_tavily_api_key")

# Search Node
def search_node(state: GraphState) -> GraphState:
    query = state["messages"][-1]["content"]
    result = search_tool.invoke(query)
    state["result"] = f"Search Result: {result}"
    state["steps"] = state.get("steps", 0) + 1
    return state

# Calculator Node
def calculator_node(state: GraphState) -> GraphState:
    query = state["messages"][-1]["content"]
    try:
        answer = str(eval(query))
    except Exception as e:
        answer = f"Error: {str(e)}"
    state["result"] = f"Calculation Result: {answer}"
    state["steps"] = state.get("steps", 0) + 1
    return state

# General Chat Node
def general_chat_node(state: GraphState) -> GraphState:
    response = llm.invoke(state["messages"])
    state["result"] = response.content
    state["steps"] = state.get("steps", 0) + 1
    return state
```

8.8 Build Graph with Conditional Routing

```
from langgraph.graph import StateGraph, END

graph = StateGraph(GraphState)

graph.add_node("router", route_query)
graph.add_node("search", search_node)
graph.add_node("calculator", calculator_node)
graph.add_node("general_chat", general_chat_node)

# Set entry point
graph.set_entry_point("router")

# Add conditional edges from router
graph.add_conditional_edges("router", route_query, {
    "search": "search",
    "calculator": "calculator",
    "general_chat": "general_chat"
})

# Connect end nodes to END
graph.add_edge("search", END)
graph.add_edge("calculator", END)
graph.add_edge("general_chat", END)

app = graph.compile()
```

8.9 Example Run and Output

```
# Run the graph
initial_state = {"messages": [{"role": "user", "content": "What is 12*5?"}]}
result = app.invoke(initial_state)
print(result["result"])
```

Output: Calculation Result: 60

8.10 Key Points

- Use conditional routing for dynamic workflows.
- You can route based on rules, LLM output or tool results.
- Keep routing logic simple and modular.
- Handle error cases and default routes.
- Can be extended to support multi-agent systems.

8.11 Summary

- Conditional routing makes LangGraph workflows more flexible.
- It helps in building real-world applications with multiple paths.
- We can combine LLMs, tools and custom logic for decision making.
- It is useful for chatbots, research assistants and multi-agent systems.

Rahul Yadav

9. Memory and Persistence in LangGraph

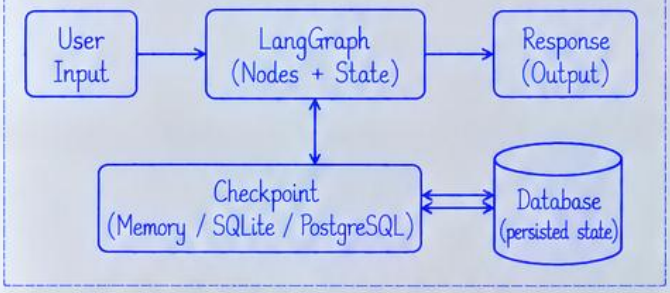
9.1 Overview

- Memory and persistence allow the graph to store conversation history, intermediate results and state across multiple runs.
- It helps in building stateful, long-running and multi-turn applications.
- LangGraph provides built-in checkpointers to persist the state in memory or external databases (e.g., SQLite, PostgreSQL).
- This is useful for chatbots, agents and complex workflows where context needs to be maintained.

9.2 Types of Memory

1. Short-term Memory - Stored in state during execution.
2. Conversation Memory - Stores chat history (messages).
3. Persistent Memory - Saves state between runs using checkpointers (SQLite, PostgreSQL, etc.).

9.3 Architecture Diagram



9.4 Checkpoint Types in LangGraph

- InMemorySaver: Stores state in memory (non-persistent).
- SQLiteSaver : Persists state in a local SQLite database.
- PostgresSaver : Persists state in PostgreSQL database.
- Custom Saver : We can implement our own checkpointer.

9.5 Example: State with Conversation Memory

```
from typing import TypedDict, List, Annotated

class GraphState(TypedDict):
    messages: Annotated[List[dict], "Conversation history"]
    user_id: str        # To identify user session
    summary: str        # Optional summary of conversation
```

9.6 Using In-Memory Checkpoint (Simple Example)

```
from langgraph.graph import StateGraph, END
from langgraph.checkpoint.memory import InMemorySaver

checkpointer = InMemorySaver()

app = graph.compile(checkpointer=checkpointer)
```

9.7 Using SQLite Checkpoint

```
from langgraph.checkpoint.sqlite import SqliteSaver

checkpointer = SqliteSaver.from_conn_string("sqlite:///langgraph.db")
app = graph.compile(checkpointer=checkpointer)
```

9.8 Complete Example (Chat with Memory)

```
from langgraph.graph import StateGraph, END
from langgraph.checkpoint.sqlite import SqliteSaver
from typing import TypedDict, List, Annotated
from langchain_openai import ChatOpenAI

class GraphState(TypedDict):
    messages: Annotated[List[dict], "Conversation history"]
    user_id: str

# Initialize LLM
llm = ChatOpenAI(model="gpt-3.5-turbo", temperature=0.0)

# Define chat node
def chat_node(state: GraphState) -> GraphState:
    response = llm.invoke(state["messages"])
    state["messages"].append(response)
    return state

# Build graph
graph = StateGraph(GraphState)
graph.add_node("chat", chat_node)
graph.set_entry_point("chat")
graph.add_edge("chat", END)

# Use SQLite checkpointer
checkpointer = SqliteSaver.from_conn_string("sqlite:///langgraph.db")
app = graph.compile(checkpointer=checkpointer)

# Run graph with a user_id to maintain separate sessions
initial_state = {
    "messages": [{"role": "user", "content": "Hello, my name is Rahul."}],
    "user_id": "user_123"
}
result = app.invoke(initial_state, config={"configurable": {"thread_id": "user_123"}})
print(result["messages"][-1]["content"])
```

9.9 Example Run and Output

```
# First run
User: Hello, my name is Rahul.
AI: Hello Rahul! Nice to meet you. How can I help you?

# Second run (same thread_id)
User: What is my name?
AI: Your name is Rahul.
```

9.10 How Persistence Works ?

1. Each run saves the state in a checkpointer (e.g., SQLite).
2. We use a thread_id (or session id) to identify the conversation.
3. On the next run, the previous state is loaded automatically.
4. This allows maintaining long-term conversation history.

9.11 Error Handling

- Handle database connection errors.
- Use try-except for LLM API failures.
- Validate state structure before saving.
- Ensure unique thread_id for each user session.

9.12 Key Points

- Use checkpointers to persist state between runs.
- SQLite is good for local development.
- PostgreSQL can be used for production.
- Thread_id helps manage multiple user sessions.
- Memory and persistence enable stateful, long-term and multi-turn applications.

Rahul Yadav

10. Human-in-the-Loop and Interaction in LangGraph

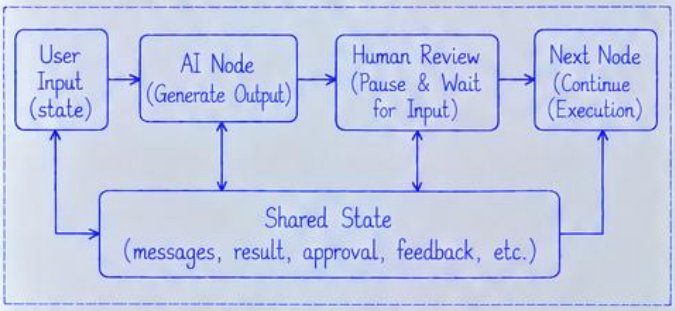
10.1 Overview

- Human-in-the-loop (HITL) allows human intervention in the graph workflow.
- It is useful when the system needs human approval, feedback or clarification.
- LangGraph makes it easy to pause the workflow, get user input and then continue execution.
- This helps in building reliable, safe and production-ready AI applications.

10.2 When to Use Human-in-the-Loop ?

- When LLM output needs human approval.
- When information is sensitive or critical.
- When the system is not confident and needs clarification.
- In multi-agent workflows where human feedback is required.
- For content review, document approval, or decision making.

10.3 Architecture Diagram



10.4 Types of Human Interaction

1. Approval - Human approves or rejects the output.
2. Feedback - Human provides feedback to improve the result.
3. Clarification - Human asks for more details.
4. Editing - Human directly edits the generated content.
5. Decision - Human selects from multiple options.

10.5 State Structure (Example)

```
from typing import TypedDict, List, Annotated

class GraphState(TypedDict):
    messages: Annotated[List[dict], "Conversation history"]
    user_input: str          # Original user query
    ai_output: str           # LLM generated output
    human_feedback: str      # Feedback from human
    is_approved: bool        # Approval status
    final_output: str        # Final result after human review
```

10.6 Human Review Node (Example)

```
def human_review_node(state: GraphState) -> GraphState:
    print("--- Human Review Required ---")
    print("AI Output:", state["ai_output"])
    feedback = input("Enter your feedback (or 'approve'): ")
    if feedback.lower() == "approve":
        state["is_approved"] = True
        state["final_output"] = state["ai_output"]

    else:
        state["is_approved"] = False
        state["human_feedback"] = feedback

    return state
```

10.7 Complete Example (HITL with Approval)

```
from langgraph.graph import StateGraph, END
from typing import TypedDict, List, Annotated
from langchain_openai import ChatOpenAI

class GraphState(TypedDict):
    messages: Annotated[List[dict], "Conversation history"]
    user_input: str
    ai_output: str
    human_feedback: str
    is_approved: bool
    final_output: str

# Initialize LLM
llm = ChatOpenAI(model="gpt-3.5-turbo", temperature=0.0)
# Node 1: Generate content
def generate_node(state: GraphState) -> GraphState:
    prompt = state["user_input"]
    response = llm.invoke(prompt)
    state["ai_output"] = response.content
    return state
# Node 2: Human review
def human_review_node(state: GraphState) -> GraphState:
    print("\n--- Human Review Required ---")
    print("AI Output:\n", state["ai_output"])
    feedback = input("Enter feedback (or type 'approve'): ")
    if feedback.lower() == "approve":
        state["is_approved"] = True
        state["final_output"] = state["ai_output"]
    else:
        state["is_approved"] = False
        state["human_feedback"] = feedback
    return state
# Node 3: Finalize output
def finalize_node(state: GraphState) -> GraphState:
    if state["is_approved"]:
        print("✅ Output approved. Proceeding to final result.")
    else:
        print("❌ Output rejected. Using human feedback.")
        state["final_output"] = f"Updated based on feedback:
                                  {state['human_feedback']}"
    return state
# Build graph
graph = StateGraph(GraphState)
graph.add_node("generate", generate_node)
graph.add_node("human_review", human_review_node)
graph.add_node("finalize", finalize_node)
graph.set_entry_point("generate")
graph.add_edge("generate", "human_review")
graph.add_edge("human_review", "finalize")
graph.add_edge("finalize", END)
app = graph.compile()
```

10.8 Example Run and Output

```
User Input: Write a short blog on LangGraph.
--- Human Review Required ---
AI Output:
   LangGraph is a powerful framework for building stateful ...
Enter feedback (or type 'approve'): approve
✅ Output approved. Proceeding to final result.
Final Output: LangGraph is a powerful framework for building ...
```

10.9 Key Points

- Human-in-the-loop improves reliability and safety.
- Use it for critical, sensitive or uncertain tasks.
- You can pause the graph and wait for human input.
- Maintain human feedback in the state.
- Useful for content review, approvals and decision making.

10.10 Summary

- Human-in-the-loop allows human intervention in LangGraph workflows.
- It helps in building more accurate, safe and real-world applications.
- You can implement approval, feedback, clarification and editing flows.
- It is widely used in enterprise AI systems.

Rahul Yadav

11. Multi-Agent Systems with LangGraph

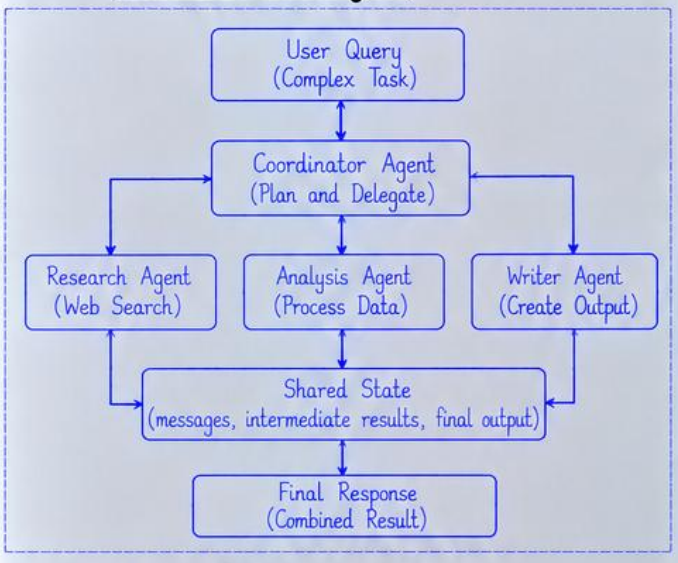
11.1 Overview

- Multi-agent systems use multiple LLM agents to work together to solve complex tasks.
- Each agent has a specific role, goal and set of tools.
- Agents can communicate, share state and delegate tasks with the help of LangGraph.
- This approach is useful for research, data analysis, content creation and real-world applications.

11.2 Key Concepts

- Agent : An LLM with a specific role and responsibility.
- Multi-Agent : Multiple agents working towards a common goal.
- Coordinator : A central agent that plans and delegates tasks.
- Communication : Agents share information through state.
- Task Decomposition : Breaking complex tasks into smaller sub-tasks.
- Role Specialization : Each agent focuses on a specific domain.

11.3 Architecture Diagram



11.4 Types of Multi-Agent Systems

1. Sequential Agents - Agents work one after another.
2. Parallel Agents - Multiple agents work at the same time.
3. Hierarchical Agents - One coordinator manages sub-agents.
4. Collaborative Agents - Agents share information and refine results together.

11.5 Define State for Multi-Agents

```
from typing import TypedDict, List, Annotated

class AgentState(TypedDict):
    messages: Annotated[List[dict], "Conversation history"]
    task: str                  # Original user task
    plan: str                  # Task breakdown plan
    research_result: str       # Output from research agent
    analysis_result: str       # Output from analysis agent
    final_result: str          # Final output
    current_agent: str         # Track which agent is running
    steps: int                 # Execution steps
```

11.6 Define Agent Nodes

- Research Agent : Searches information using web tools.
- Analysis Agent : Analyzes and processes the data.
- Writer Agent : Creates the final response.
- Coordinator Agent: Plans the task and assigns work to agents.

11.7 Example: Research Agent (With Tool)

```
from langgraph.graph import StateGraph
from langchain_openai import ChatOpenAI
from langchain_community.tools import TavilySearchResults
search_tool = TavilySearchResults(api_key="your_tavily_api_key",
                                  max_results=3)
llm = ChatOpenAI(model="gpt-3.5-turbo", temperature=0.0)
def research_agent(state: AgentState) -> AgentState:
    query = state["task"]
    search_results = search_tool.invoke(query)
    summary = llm.invoke(f"Summarize the following results:
        {search_results}")
    state["research_result"] = summary.content
    state["current_agent"] = "research"
    return state
```

11.8 Example: Analysis Agent

```
def analysis_agent(state: AgentState) -> AgentState:
    research_data = state["research_result"]
    prompt = f"Analyze the following information and give
    key insights:\n{research_data}"
    result = llm.invoke(prompt)
    state["analysis_result"] = result.content
    state["current_agent"] = "analysis"
    return state
```

11.9 Example: Writer Agent

```
def writer_agent(state: AgentState) -> AgentState:
    task = state["task"]
    analysis = state["analysis_result"]
    prompt = f"Write a detailed and well-structured answer
    for the task: {task}\n using the analysis: {analysis}"
    result = llm.invoke(prompt)
    state["final_result"] = result.content
    state["current_agent"] = "writer"
    return state
```

11.10 Build Multi-Agent Graph

```
graph = StateGraph(AgentState)
graph.add_node("research", research_agent)
graph.add_node("analysis", analysis_agent)
graph.add_node("writer", writer_agent)
graph.set_entry_point("research")
graph.add_edge("research", "analysis")
graph.add_edge("analysis", "writer")
graph.add_edge("writer", END)

app = graph.compile()

# Run the graph
initial_state = {"task": "Explain the applications of AI in banking",
                 "messages": [], "steps": 0}
result = app.invoke(initial_state)
print(result["final_result"])
```

11.11 Example Output (Simplified)

```
Research Agent: Found latest information about AI in banking.
Analysis Agent: Identified key use cases and benefits.
Writer Agent: Generated final response.

Final Output:
AI is widely used in banking for fraud detection, risk assessment,
customer support, personalized financial products, ...
```

11.12 Key Points

- Use multiple specialized agents for complex tasks.
- Maintain shared state for communication between agents.
- You can use tools, memory and human-in-the-loop with multi-agents.
- Design clear roles and prompts for each agent.
- Handle errors and retries for better reliability.

11.13 Summary

- LangGraph makes it easy to build multi-agent systems.
- Agents can work together to solve complex real-world problems.
- This enables powerful applications like research assistants, data analysis pipelines and enterprise AI systems.

Rahul Yadav

12. Advanced Graph Patterns in LangGraph

12.1 Overview

- Advanced graph patterns help in building complex, flexible and production-ready workflows.
- These patterns are useful for real-world applications like research assistants, multi-step data pipelines, agentic systems, etc.
- LangGraph supports various patterns such as loops, parallel execution, subgraphs, dynamic routing, retry logic and map-reduce.
- We can combine multiple patterns to create powerful and scalable applications.

12.2 Common Advanced Patterns

1. Sequential Pattern - Nodes execute one after another.
2. Parallel Pattern - Multiple nodes run in parallel.
3. Conditional Pattern - Route based on state or LLM output.
4. Loop Pattern - Repeat a set of nodes until a condition.
5. Map-Reduce Pattern - Process multiple items and combine results.
6. Subgraph Pattern - Reuse a group of nodes as a subgraph.
7. Dynamic Graph Pattern - Create graph dynamically at runtime.
8. Retry Pattern - Retry on failure with error handling.

12.3 Sequence Pattern (Example)

- Used for simple linear workflows.
- Each node executes after the previous node.

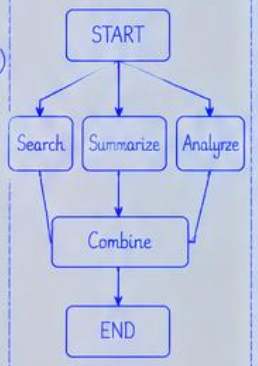
```
from langgraph.graph import StateGraph, END
graph = StateGraph(dict)
graph.add_node("step1", step1_node)
graph.add_node("step2", step2_node)
graph.add_edge("step1", "step2")
graph.add_edge("step2", END)
app = graph.compile()
```



12.4 Parallel Pattern (Example)

- Run multiple nodes at the same time.
- Useful for independent tasks like web search, data fetching, etc.

```
graph = StateGraph(dict)
graph.add_node("search", search_node)
graph.add_node("summarize", summarize_node)
graph.add_node("analyze", analyze_node)
graph.add_edge(START, "search")
graph.add_edge(START, "summarize")
graph.add_edge(START, "analyze")
graph.add_edge("search", "combine")
graph.add_edge("summarize", "combine")
graph.add_edge("analyze", "combine")
graph.add_node("combine", combine_node)
graph.add_edge("combine", END)
app = graph.compile()
```

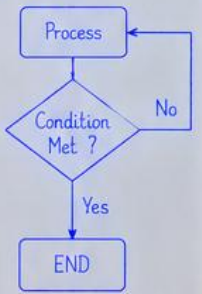


12.5 Loop Pattern (Example)

- Repeat a set of nodes until a condition is met.
- Useful for iterative research, refinement or validation.

```
def should_continue(state):
    return state["iteration"] < 3

graph.add_conditional_edges(
    "process", should_continue,
    {"continue": "process", "end": END}
)
```

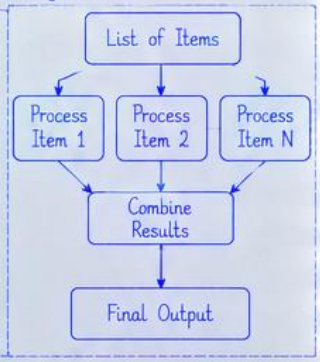


12.6 Map-Reduce Pattern (Example)

- Used to process multiple items in parallel and then combine the results.
- Useful for document processing, data analysis, etc.

```
# Map Node - Process each item
def map_node(state):
    results = []
    for item in state["items"]:
        result = process_item(item)
        results.append(result)
    return {"map_results": results}

# Reduce Node - Combine all results
def reduce_node(state):
    combined = " ".join(state["
    "map_results"])
    return {"final_output": combined}
```

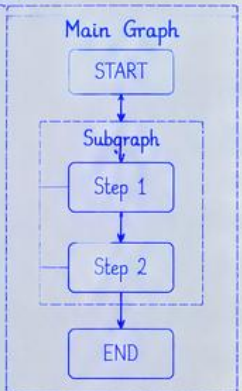


12.7 Subgraph Pattern (Example)

- Used to group a set of nodes into a subgraph.
- Helps in reusability and better organization.

```
# Create a subgraph
subgraph = StateGraph(dict)
subgraph.add_node("step1", step1_node)
subgraph.add_node("step2", step2_node)
subgraph.add_edge("step1", "step2")
subgraph_app = subgraph.compile()

# Use subgraph in main graph
graph = StateGraph(dict)
graph.add_node("subgraph", subgraph_app)
graph.add_edge(START, "subgraph")
graph.add_edge("subgraph", END)
app = graph.compile()
```



12.8 Dynamic Graph Pattern (Example)

- Create nodes and edges dynamically at runtime.
- Useful when workflow depends on user input or external data.

```
graph = StateGraph(dict)
for step in steps_from_config:
    graph.add_node(step["name"], step["func"])
for i in range(len(steps_from_config) - 1):
    graph.add_edge(steps_from_config[i]["name"],
                   steps_from_config[i+1]["name"])
graph.add_edge(steps_from_config[-1]["name"], END)
app = graph.compile()
```

12.9 Retry Pattern (Example)

- Retry a node when it fails (e.g., API error, timeout).
- Useful for production systems to handle transient failures.

```
from tenacity import retry, stop_after_attempt, wait_fixed
@retry(stop=stop_after_attempt(3), wait=wait_fixed(2))
def api_node(state):
    try:
        result = call_external_api()
        return {"api_result": result}
    except Exception as e:
        return {"error": str(e)}
```

12.10 Key Points

- Use the right pattern based on the problem.
- Combine multiple patterns for complex workflows.
- Keep the graph modular using subgraphs.
- Handle errors and retries in production.
- Advanced patterns make LangGraph suitable for real-world applications.

12.11 Summary

- LangGraph supports many advanced patterns.
- These patterns help in building scalable, flexible and production-ready agentic systems.
- We can use loops, parallel execution, subgraphs, map-reduce and dynamic graphs to solve complex tasks efficiently.

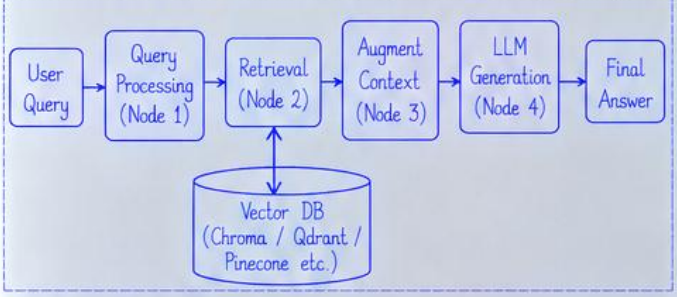
Rahul Yadav

13. RAG with LangGraph

13.1 Overview

- RAG (Retrieval Augmented Generation) combines retrieval of relevant information from external sources with LLM generation.
- LangGraph helps in building stateful, multi-step RAG pipelines with better control, routing and memory.
- Useful for document Q&A, knowledge bases, chatbots and enterprise applications.

13.2 RAG Architecture



13.3 Key Components

1. Document Loading - Load data from PDF, text, web, etc.
2. Text Splitting - Split documents into chunks.
3. Embedding - Convert text to vector embeddings.
4. Vector Store - Store and search embeddings (Chroma, Qdrant, Pinecone, etc.).
5. Retriever - Find relevant chunks for a query.
6. Augmentation - Create context with retrieved data.
7. LLM Generation - Generate final answer using LLM.
8. State Management - Maintain query, context, results, etc.

13.4 Graph State Definition

```
from typing import TypedDict, List, Annotated

class RAGState(TypedDict):
    query: str                  # User question
    retrieved_docs: List[str]   # Relevant documents
    context: str                # Combined context
    answer: str                 # Final LLM answer
    sources: List[str]          # Source information
    steps: int                  # Track execution steps
```

13.5 Document Loading and Vector Store

```
from langchain_community.document_loaders import PyPDFLoader
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain_community.embeddings import
   HuggingFaceEmbeddings
from langchain_community.vectorstores import Chroma

# Load PDF document
loader = PyPDFLoader("data/sample.pdf")
documents = loader.load()

# Split into chunks
splitter = RecursiveCharacterTextSplitter(
    chunk_size=1000, chunk_overlap=200
)
chunks = splitter.split_documents(documents)

# Create embeddings and store in Chroma
embeddings = HuggingFaceEmbeddings(model_name="all-MiniLM-L6-v2")
vector_db = Chroma.from_documents(
    documents=chunks, embedding=embeddings,
)   persist_directory="./chroma_db"
```

13.6 Define RAG Nodes

```
# Node 1: Retrieve relevant documents
def retrieve_node(state: RAGState) -> RAGState:
    query = state["query"]
    retriever = vector_db.as_retriever(search_kwargs={"k": 4})
    docs = retriever.invoke(query)
    state["retrieved_docs"] = [d.page_content for d in docs]
    state["sources"] = [d.metadata.get("source", "") for d in docs]
    return state
# Node 2: Create context from retrieved docs
def augment_node(state: RAGState) -> RAGState:
    docs = state["retrieved_docs"]
    context = "\n\n".join([f"[Doc {i+1}]: {doc}"
                           for i, doc in enumerate(docs)])
    state["context"] = context
    return state
# Node 3: Generate answer using LLM
from langchain_openai import ChatOpenAI
llm = ChatOpenAI(model="gpt-3.5-turbo", temperature=0.0)
def generate_node(state: RAGState) -> RAGState:
    prompt = f"""
    Use the following context to answer the question.
    If the answer is not in the context, say you don't know.
    \nContext:\n{state['context']}\n\n
    Question: {state["query"]}\n\n
    Answer:
    """
    response = llm.invoke(prompt)
    state["answer"] = response.content
    state["steps"] = state.get("steps", 0) + 1
    return state
```

13.7 Build RAG Graph

```
from langgraph_graph import StateGraph, END
graph = StateGraph(RAGState)
graph.add_node("retrieve", retrieve_node)
graph.add_node("augment", augment_node)
graph.add_node("generate", generate_node)
graph.set_entry_point("retrieve")
graph.add_edge("retrieve", "augment")
graph.add_edge("augment", "generate")
graph.add_edge("generate", END)
app = graph.compile()
```

13.8 Run Example

```
initial_state = {
    "query": "What is RAG and how does it work?",
    "steps": 0
}
result = app.invoke(initial_state)
print("Answer:\n", result["answer"])
print("\nSources:", result["sources"])
```

13.9 Example Output (Simplified)

```
Answer:
RAG (Retrieval Augmented Generation) combines information
retrieval from external sources with LLM generation. It first
retrieves relevant documents based on the query and then uses
those documents as context to generate a more accurate answer.

Sources:
['rag_introduction.pdf', 'vector_database.pdf', 'langgraph_guide.pdf']
```

13.10 Key Points

- Use good text splitting and chunk size.
- Choose the right embedding model and vector store.
- Retrieve top-k relevant documents.
- Use clear prompts to reduce hallucinations.
- LangGraph helps in building modular and stateful RAG pipelines.
- Useful for enterprise knowledge bases and document Q&A systems.

13.11 Summary

- RAG with LangGraph enables reliable and accurate LLM applications.
- It combines retrieval, context augmentation and generation in a graph.
- It can be extended with tools, memory, multi-agents and human-in-the-loop.

Rahul Yadav

14. Practical Projects with LangGraph

14.1 Overview

- Practical projects help in understanding how LangGraph can be used in real-world scenarios.
- We combine LLMs, tools, memory, RAG and multi-agent patterns to build useful applications.
- These projects can be extended and customized based on your use case.
- Good for portfolio, learning and interview preparation.

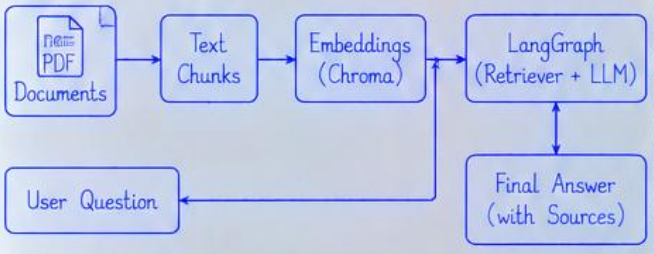
14.2 List of Projects Covered

1. Document Q&A System (RAG)
2. Web Research Assistant (with tools)
3. Multi-Agent Research and Report Generator
4. Customer Support Chatbot (with memory)
5. Data Analysis Assistant (using Python tool)
6. Email Drafting Assistant (with human review)

14.3 Project 1: Document Q&A System (RAG)

- Upload PDF documents and ask questions.
- Uses LangGraph, document loader, embeddings and vector DB.
- Returns accurate answers with document sources.

Architecture Diagram



Code (Simplified)

```
from langgraph.graph import StateGraph, END
from typing import TypedDict, List, Annotated

class RAGState(TypedDict):
    query: str              # User question
    docs: List[str]         # Retrieved documents
    answer: str             # Final answer
    sources: List[str]      # Document sources
)
# define nodes: retrieve -> generate (similar to previous page)
graph = StateGraph(RAGState)
graph.add_node("retrieve", retrieve_node)
graph.add_node("generate", generate_node)
graph.set_entry_point("retrieve")
graph.add_edge("retrieve", "generate")
graph.add_edge("generate", END)
app = graph.compile()
```

14.4 Project 2: Web Research Assistant

- Uses web search tool to get latest information.
- Can visit multiple sources, summarize and provide final answer.
- Useful for research, news and market analysis.

Tools Used

Tavily Search / SerpAPI	- Web search
Requests	- Fetch web content
LangGraph	- Workflow and routing
LLM	- Summarization

Architecture



Code (Simplified)

```
from langchain_community.tools import TavilySearchResults
search_tool = TavilySearchResults(api_key="your_api_key", max_results=3)

def search_node(state):
    query = state["query"]
    results = search_tool.invoke(query)
    state["search_results"] = results
    return state

# Add summarization node and build graph similar to RAG project
```

14.5 Project 3: Multi-Agent Research & Report Generator

- Uses multiple agents (research, analysis, writing).
- Coordinator agent manages the workflow.
- Generates a well-structured report on any topic.

Architecture Diagram



Example Use Case

- Input: "Write a report on electric vehicles market in India."
- Research Agent: Collect latest information.
- Analysis Agent: Analyze key trends and numbers.
- Writer Agent: Create a structured report with insights.

14.6 Project 4: Customer Support Chatbot

- Uses conversation memory to maintain context.
- Can answer FAQs, check order status, and create support tickets.
- Can use tools to fetch data from internal systems (e.g., API, DB).

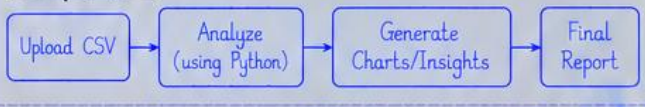
Features

- Conversation memory (InMemorySaver / SQLite)
- FAQ retrieval using RAG
- Tool to fetch user/order details
- Human handover for complex issues

14.7 Project 5: Data Analysis Assistant

- Uses Python REPL tool or code execution tool.
- Accepts data files (CSV/Excel), performs analysis and visualization.
- Useful for business analytics and quick data insights.

Example Flow



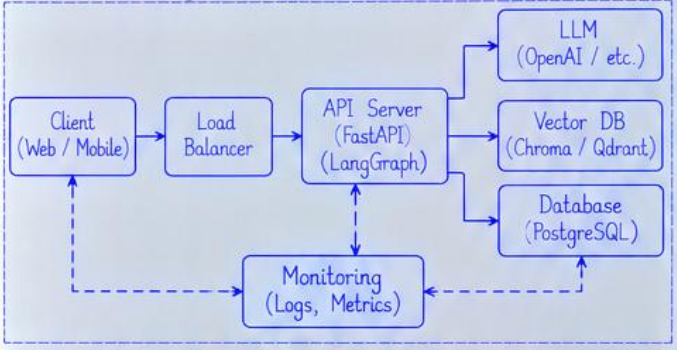
Rahul Yadav

15. Deployment and Production Setup

15.1 Overview

- After building a LangGraph application, we need to deploy it for real users.
- Deployment makes the application accessible, scalable and reliable.
- We can deploy using FastAPI, Docker, cloud services and monitoring tools.
- In production, we also need to handle logging, error handling, security, scaling and persistent storage.

15.2 Deployment Architecture



15.3 Create FastAPI for LangGraph

- We can expose our LangGraph workflow as a REST API using FastAPI.
- This allows us to send user input and get the final result.
- We can also maintain session_id for conversation memory.

```
from fastapi import FastAPI
from pydantic import BaseModel
from langgraph.graph import StateGraph, END

app = FastAPI(title="LangGraph API")

class ChatRequest(BaseModel):
    message: str
    session_id: str = "default"
# assume we already created graph
# graph = graph.compile()

@app.post("/chat")
async def chat(req: ChatRequest):
    initial_state = {
        "messages": [req.message],
        "session_id": req.session_id
    }
    result = graph.invoke(initial_state)
    return {"answer": result["answer"]}
```

15.4 Run and Test Locally

- Install required packages and run FastAPI server.
- We can test the API using Swagger UI or Postman.

```
# Install
pip install fastapi uvicorn

# Run server
uvicorn app:app --reload --host 0.0.0.0 --port 8000

# Open in browser
http://localhost:8000/docs  # Swagger UI
```

15.5 Dockerization

- Docker helps to create a portable environment for our application.
- We can package the FastAPI + LangGraph app with all dependencies.

```
# Dockerfile
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
CMD ["uvicorn", "app:app", "--host", "0.0.0.0", "--port", "8000"]
```

15.6 requirements.txt (Example)

```
fastapi
uvicorn
langgraph
langchain
openai
chromadb
pydantic
python-dotenv
```

15.7 Build and Run Docker Container

- Build the Docker image and run the container.
- We can expose the application on any port.

```
# Build image
docker build -t langgraph-app .

# Run container
docker run -p 8000:8000 --env-file .env langgraph-app
```

15.8 Environment Variables (.env)

- Keep sensitive information like API keys in .env file.
- Never hardcode keys in code.

```
OPENAI_API_KEY=your_openai_api_key
VECTOR_DB_PATH=./chroma_db
DATABASE_URL=postgresql://user:password@localhost:5432/langgraph
```

15.9 Deploy on Cloud (Options)

1. Render - Easy and free tier available.
2. Railway - Simple deployment with GitHub integration.
3. AWS (EC2/ECS) - Production ready and scalable.
4. Google Cloud Run - Serverless deployment.
5. Azure App Service - Good for enterprise applications.
6. Kubernetes - For large scale and high availability.

15.10 Production Setup Considerations

1. Use a persistent vector database (Chroma, Qdrant, Pinecone).
2. Use PostgreSQL for storing conversation history and metadata.
3. Enable logging and monitoring (Prometheus + Grafana).
4. Implement authentication and rate limiting.
5. Use a load balancer for high availability.
6. Handle exceptions and retries gracefully.
7. Set up CI/CD for automatic deployment.

15.11 Monitoring and Logging

- Use logging to track requests, errors and performance.
- Tools: Prometheus, Grafana, Loki, or cloud monitoring services.

```
import logging
logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s - %(levelname)s - %(message)s"
)

logger = logging.getLogger(__name__)

@app.post("/chat")
async def chat(req: ChatRequest):
    logger.info(f"New request: {req.message}")
    try:
        result = graph.invoke({"messages": [req.message]})
        logger.info("Request processed successfully")
        return {"answer": result["answer"]}
    except Exception as e:
        logger.error(f"Error: {str(e)}")
        return {"error": "Internal Server Error"}
```

15.12 Summary

- We can deploy LangGraph applications using FastAPI and Docker.
- Use persistent storage, monitoring and authentication for production.
- Choose the right cloud platform based on your needs.
- Follow best practices for security, scalability and reliability.
- Deployment makes the application accessible to real users.

Rahul Yadav

16. Evaluation and Debugging in LangGraph

16.1 Overview

- Evaluation and debugging are important to build reliable LangGraph applications.
- Evaluation helps us measure the quality, accuracy and relevance of LLM responses.
- Debugging helps us find and fix issues in graph flow, node execution, tool usage and state management.
- We can use logs, tracing, test datasets and evaluation metrics to improve our application.

16.2 Why Evaluation is Important ?

- LLM responses can be inconsistent.
  We need to check if the answer is correct, relevant and safe.
- Helps in comparing different prompts, models and retrieval methods.
- Useful for production to maintain quality and reliability.

16.3 Evaluation Metrics

1. Relevance - Is the answer related to the question ?
2. Correctness - Is the answer factually correct ?
3. Context Utilization - Does it use the provided context ?
4. Conciseness - Is the answer clear and to the point ?
5. Groundedness - Is the answer supported by sources ?
6. Safety - Does it avoid harmful or sensitive content ?

16.4 Evaluation Workflow



16.5 Create Evaluation Dataset (Example)

- We can create a simple dataset with questions, expected answers and relevant documents.
- This can be a CSV or JSON file.

```
[
  {  "question": "What is LangGraph?",
     "expected_answer": "LangGraph is a library to build stateful...",
     "source": "langgraph_intro.pdf"
  },

  {  "question": "Explain RAG.",
     "expected_answer": "RAG combines retrieval with LLM...",
     "source": "rag_guide.pdf"
  }
]
```

16.6 Run Evaluation with LLM (Example)

- We can use another LLM or evaluation model to score the response.
- We can define custom evaluation functions.

```
from openai import OpenAI
client = OpenAI(model="gpt-3.5-turbo")

def evaluate_answer(question, expected, actual):
    prompt = f"""
        Compare the expected answer and actual answer.
        Give a score from 0 to 1 for correctness and relevance.
        Question: {question}
        Expected: {expected}
        Actual: {actual}
        Return only in JSON format: {"relevance": x, "correctness": y}
        """

        response = client.chat.completions.create(
            messages=[{"role": "user", "content": prompt}]
        )
        return response.choices[0].message.content
```

16.7 Example: Evaluate LangGraph RAG App

```
import json
from dataset import load_dataset  # your test dataset

results = []
for item in test_dataset:
    question = item["question"]
    expected = item["expected_answer"]
    response = graph.invoke({"query": question})
    actual = response["answer"]
    eval_result = evaluate_answer(question, expected, actual)
    results.append({
        "question": question,
        "answer": actual,
        "evaluation": eval_result
    })

# Save results
def save_results(results, filename="evaluation_results.json"):
    with open(filename, "w") as f:
        json.dump(results, f, indent=2)
save_results(results)
```

16.8 Debugging in LangGraph

- We can debug node execution, state changes, tool calls and errors.
- LangGraph provides tracing and logging to understand the flow.
- We can also print state at each node or use custom logs.

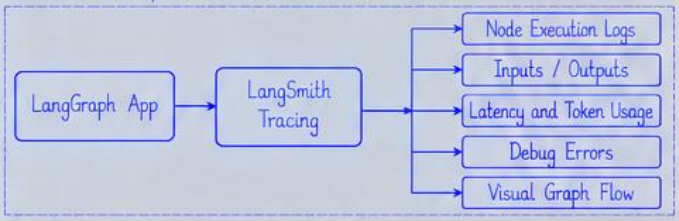
16.9 Enable Debugging and Tracing

```
from langgraph.graph import StateGraph, END
import logging

logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s - %(levelname)s - %(message)s"
)
# Add debug logs in nodes
def retrieve_node(state):
    logging.info(f"[Retrieve] Query: {state['query']}")
    # retrieval logic
    logging.info(f"[Retrieve] Retrieved docs: {len(docs)}")
    return {"docs": docs}
```

16.10 Using LangSmith for Tracing

- LangSmith helps in tracing, debugging and monitoring LangGraph apps.
- It shows each node execution, inputs, outputs, token usage and latency.
- Useful for production and team collaboration.



16.11 Common Issues and Solutions

Issue	Reason	Solution
Node not executed	Wrong edge connection	Check graph flow and edges
Empty response	Poor prompt / no context	Improve prompt and retrieval
Tool failure	API error / wrong input	Add error handling and retries
State not updated	Wrong key name	Check state keys in each node
Incorrect output	Irrelevant documents	Use better chunking and reranking
High latency	Large context / model	Optimize context and use faster model

16.12 Key Points

- Always test with a good dataset.
- Use evaluation metrics to measure quality.
- Enable logging and tracing for debugging.
- Use LangSmith for better observability.
- Handle errors and edge cases properly.
- Keep improving prompts, retrieval and model selection.

16.13 Summary

- Evaluation and debugging help us build reliable and high-quality LangGraph applications.
- We can use logs, tracing, test datasets and metrics to find issues and improve performance.
- LangSmith makes it easy to visualize and debug complex graphs.

Rahul Yadav

17. Best Practices and Optimization in LangGraph

17.1 Overview

- Following best practices helps in building reliable, efficient and scalable LangGraph applications.
- Optimization improves performance, reduces cost and latency, and enhances user experience.
- We should consider prompt design, graph design, memory management, model selection, caching and monitoring.
- These practices are useful for production-grade applications and real-world use cases.

17.2 Best Practices for Graph Design

1. Keep the graph modular using small and focused nodes.
2. Use clear node names and meaningful state variables.
3. Handle errors and retries properly.
4. Keep the graph simple and easy to maintain.
5. Use subgraphs for reusability and complex workflows.
6. Avoid circular dependencies and unnecessary loops.
7. Add logging and tracing for better observability.
8. Validate inputs and handle edge cases.

17.3 Prompt Engineering Best Practices

- Use clear and specific instructions in prompts.
- Provide context and constraints.
- Use few-shot examples when needed.
- Keep prompts short and focused.
- Use structured output formats (JSON) for better results.
- Avoid ambiguous instructions.
- Test different prompt templates and compare results.

17.4 Model Selection and Configuration

- Choose the right model based on the use case.
- Consider cost, latency, and accuracy.
- Use smaller models for simple tasks.
- Use temperature, max_tokens and other parameters wisely.
- Use model fallbacks in case the primary model fails.

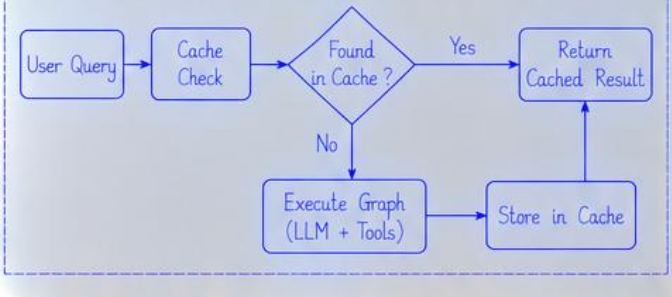
```
from langchain_openai import ChatOpenAI

# GPT-4 for complex reasoning
llm = ChatOpenAI(model="gpt-4", temperature=0.2)

# GPT-3.5 for simple tasks (low cost)
fast_llm = ChatOpenAI(model="gpt-3.5-turbo", temperature=0.0,
           max_tokens=500)
```

17.5 Caching and Performance Optimization

- Use caching to reduce LLM API calls and cost.
- Cache embeddings and retrieval results.
- Use vector database for efficient search.
- Implement response caching for repeated queries.
- Use streaming for faster user experience.
- Parallelize independent nodes where possible.
- Optimize chunking strategy for better retrieval.



17.6 Memory Management Best Practices

- Use appropriate memory type (in-memory, SQLite, PostgreSQL).
- Use thread_id to maintain conversation history.
- Store only necessary information in state.
- Clean old conversation data when not needed.
- Use summarization for long conversations.

```
from langgraph.checkpoint.sqlite import SqliteSaver
memory = SqliteSaver.from_conn_string("checkpoints.db")
graph = builder.compile(checkpointer=memory)
config = {"configurable": {"thread_id": "user123"}}
result = graph.invoke(initial_state, config=config)
```

17.7 Error Handling and Retries

- Use try-except blocks in each node.
- Implement retry logic for API failures.
- Handle tool failures and invalid outputs.
- Return meaningful error messages.
- Use fallback nodes when a node fails.
- Log errors for debugging and monitoring.

```
from tenacity import retry, stop_after_attempt, wait_exponential

@retry(stop=stop_after_attempt(3), wait=wait_exponential(min=1, max=10))
def call_llm(prompt):
    try:
        return llm.invoke(prompt)
    except Exception as e:
        print(f"Error: {str(e)}")
        return ""
```

17.8 Cost Optimization Tips

- Use smaller models for simple tasks.
- Cache responses and embeddings.
- Avoid unnecessary tool calls.
- Use streaming to improve user experience.
- Limit the number of tokens in prompts and responses.
- Batch multiple requests when possible.
- Monitor usage with LangSmith or cloud provider dashboard.

17.9 Scalability and Production Readiness

- Use FastAPI with multiple workers.
- Deploy using Docker and orchestrate with Kubernetes.
- Use a persistent database for memory and metadata.
- Implement authentication and rate limiting,
- Use monitoring tools (Prometheus, Grafana, LangSmith).
- Set up alerts for errors and high latency.

17.10 Example: Optimized Node with Caching and Retry

```
from functools import lru_cache

@lru_cache(maxsize=100)
def get_embedding(text):
    return embeddings_model.embed_query(text)

@retry(stop=stop_after_attempt(3), wait=wait_exponential(min=1, max=5))
def retrieve_node(state):
    query = state["query"]
    docs = vectorstore.similarity_search(query, k=4)
    return {"docs": docs}
```

17.11 Key Points

- Follow modular and clean graph design.
- Use proper prompt engineering and model selection.
- Implement caching and retry logic.
- Monitor, log and optimize performance.
- Make the application scalable and production ready.

17.12 Summary

- Best practices and optimization help in building robust, efficient and scalable LangGraph applications.
- They reduce cost, improve performance and enhance reliability.
- These practices are essential for real-world and production use cases.

Rahul Yadav

18. Common Challenges and Solutions in LangGraph

18.1 Overview

- While building LangGraph applications, we may face various challenges related to graph design, LLM behavior, tools, memory, performance and production.
- Understanding common issues and their solutions helps in building reliable and robust applications.
- Here we cover the most common challenges with practical solutions and examples.

18.2 Issue: Incorrect or Hallucinated Answers

- Problem: LLM generates incorrect, irrelevant or hallucinated answers.
- Causes: Poor retrieval, low quality documents, weak prompts.
- Solutions:
  1. Use better document chunking and embeddings.
  2. Provide clear prompt with context and constraints.
  3. Return source documents with answers.
  4. Validate answer using another LLM (optional).

Example: Use strict prompt with context

```
prompt = f"""
    Answer the question using only the provided context.
    If the answer is not in the context, say "I don't know".

    Context: {context}

    Question: {question}
    Answer:"""
```

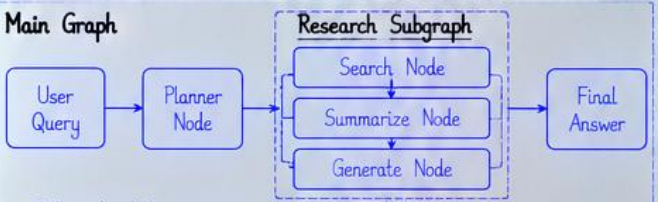
18.3 Issue: Poor Retrieval Results

- Problem: Relevant documents are not retrieved.
- Causes: Wrong embedding model, small chunk size, incorrect search parameters.
- Solutions:
  1. Use high quality embedding model (e.g., all-MiniLM-L6-v2, OpenAI embeddings).
  2. Optimize chunk size (500-1000 tokens).
  3. Use metadata filters and hybrid search (vector + keyword).
  4. Try different similarity search parameters (k, score threshold).

18.4 Issue: Graph Complexity and Maintenance

- Problem: Graph becomes complex and hard to maintain.
- Causes: Too many nodes, unclear flow, tight coupling.
- Solutions:
  1. Keep graph modular using subgraphs.
  2. Use clear node names and state structure.
  3. Separate business logic into individual functions.
  4. Use configuration for dynamic graph building.

18.5 Example: Using Subgraph for Reusability



- Subgraphs help in reusing common workflows.
- It keeps the main graph clean and easy to maintain.

18.6 Issue: Memory and State Management

- Problem: Conversation history is lost or state is not updated correctly.
- Causes: Not using checkpoint, wrong memory type, missing thread_id.
- Solutions:
  1. Use InMemorySaver for development and PostgreSQL/SQLite for production.
  2. Pass thread_id to maintain conversation history.
  3. Clean old memory when it grows too large.
  4. Store only necessary information in state.

18.7 Example: Using Memory with Checkpoint

```
from langgraph.checkpoint.sqlite import SQLiteSaver

memory = SQLiteSaver.from_conn_string("checkpoints.db")
graph = builder.compile(checkpointer=memory)

config = {"configurable": {"thread_id": "user123"}}
result = graph.invoke({"query": "Hello"}, config=config)
```

18.8 Issue: Tool Failures and API Errors

- Problem: External tools (search, API, DB) fail or return errors.
- Causes: Network issues, invalid API key, rate limits, timeout.
- Solutions:
  1. Use try-except blocks in tool functions.
  2. Implement retry logic with exponential backoff.
  3. Handle error gracefully and provide fallback response.
  4. Log tool errors for debugging.

Example: Tool with Retry Logic

```
from tenacity import retry, stop_after_attempt, wait_exponential

@retry(stop_after_attempt=3, wait=wait_exponential(min=1, max=10))
def search_tool(query: str):
    try:
        # call external API
        return perform_search(query)
    except Exception as e:
        return {"error": str(e)}
```

18.9 Issue: High Latency and Slow Responses

- Problem: Application takes too long to respond.
- Causes: Large documents, multiple tool calls, big models.
- Solutions:
  1. Use smaller and faster models for simple tasks.
  2. Implement caching for embeddings and LLM responses.
  3. Use parallel execution for independent nodes.
  4. Optimize chunk size and retrieval parameters.
  5. Stream responses to improve user experience.

18.10 Issue: High Cost

- Problem: API cost becomes high due to large prompts and many requests.
- Solutions:
  1. Use smaller models (e.g., gpt-3.5-turbo) for simple tasks.
  2. Cache results and avoid repeated calls.
  3. Limit the number of retrieved documents.
  4. Use streaming and early stopping.
  5. Compress and summarize large context.

18.11 Issue: Deployment and Production Errors

- Problem: Application fails in production environment.
- Causes: Environment misconfiguration, missing dependencies, resource limits, invalid API keys.
- Solutions:
  1. Use environment variables for all sensitive keys.
  2. Test locally using Docker before deploying.
  3. Set up proper logging, monitoring and alerts.
  4. Use health checks and retry policies.
  5. Ensure database and storage are properly configured.

18.12 Key Points

- Use clear and structured prompts.
- Optimize retrieval and embeddings.
- Keep graph modular and maintainable.
- Handle errors and retries properly.
- Use caching to reduce cost and latency.
- Manage memory and state correctly.
- Test thoroughly before production.
- Monitor logs and performance.
- Follow best practices for security, scalability and reliability.

18.13 Summary

- LangGraph applications can face various challenges, but with the right strategies and best practices we can solve them effectively.
- Proper graph design, prompt engineering, error handling, caching, monitoring and optimization help in building robust and production-ready applications.

Rahul Yadav

19. Interview Questions and Final Revision

19.1 Overview

- This page contains important interview questions, key concepts and a final revision checklist for LangGraph.
- Covers conceptual, coding, architecture and real-world scenario based questions.
- Helpful for interviews, assessments and quick revision.
- Use this page to revise before your interview.

19.2 Conceptual Questions

1. What is LangGraph and how is it different from LangChain ?
2. What are the key components of LangGraph ?
3. How does state management work in LangGraph ?
4. What are nodes and edges in a graph ?
5. When should we use LangGraph instead of a simple chain ?
6. How does LangGraph support multi-agent workflows ?
7. What is human-in-the-loop and why is it useful ?
8. How can we make LangGraph applications production ready ?
9. How to handle errors and retries in LangGraph ?
10. What are the common use cases of LangGraph ?

19.3 Coding Questions

1. Create a simple LangGraph with two nodes (generate and review).
2. Write a conditional node to decide next step based on state.
3. Implement a loop to retry a node on failure.
4. Create a subgraph and reuse it in main graph.
5. Build a RAG pipeline using LangGraph (retrieve → generate).
6. Implement a multi-agent graph (research → analyze → write).
7. Add human approval step in the workflow.
8. Save and load state using checkpoint (SQLite).
9. Implement caching for embeddings and retrieval results.
10. Add streaming response to LangGraph application.

19.4 Example Coding Question

Q: Create a LangGraph with conditional routing based on user input.

```
from typing import TypedDict
from langgraph.graph import StateGraph, END

class GraphState(TypedDict):
    question: str
    route: str
    answer: str

def route_node(state):
    q = state["question"]
    if "weather" in q.lower():
        return {"route": "weather"}
    else:
        return {"route": "general"}

def weather_node(state):
    return {"answer": "Fetching weather information..."}

def general_node(state):
    return {"answer": "This is a general response."}

graph = StateGraph(GraphState)
graph.add_node("route", route_node)
graph.add_node("weather", weather_node)
graph.add_node("general", general_node)
graph.add_conditional_edges(
    "route",
    lambda x: x["route"],
    {"weather": "weather", "general": "general"}
)
graph.add_edge("weather", END)
graph.add_edge("general", END)

app = graph.compile()
```

19.5 Architecture Questions

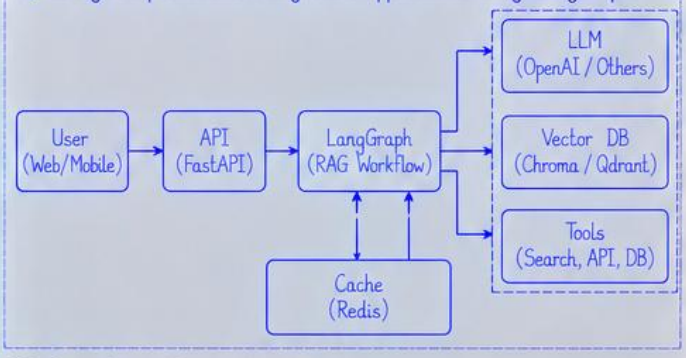
1. How would you design a scalable LangGraph application ?
2. How do you integrate external tools (search, API, DB) ?
3. How do you manage conversation memory in production ?
4. How would you handle high traffic and rate limits ?
5. Which vector database would you choose and why ?
6. How do you ensure reliability and fault tolerance ?
7. How would you deploy LangGraph using Docker and cloud ?
8. How do you monitor and debug LangGraph applications ?
9. How can you reduce latency and cost ?
10. How do you secure APIs and sensitive data ?

19.6 Scenario Based Questions

1. RAG chatbot is giving irrelevant answers. How will you improve it ?
2. API is failing intermittently. How will you handle it ?
3. You need to add human approval in the workflow. How will you design it ?
4. Application is slow for large documents. What optimizations will you use ?
5. How will you design a multi-agent research assistant ?
6. You need to deploy the application for 1M users. What will you consider ?
7. How will you evaluate the quality of responses ?
8. Tool call is returning an error. How will you handle retries ?
9. You need to maintain conversation history across sessions. How ?
10. How will you reduce cost while maintaining good accuracy ?

19.7 System Design Question (Example)

Q: Design a production-ready RAG application using LangGraph.



19.8 Key Topics to Revise

- LangGraph core concepts
- Graph design patterns
- State management
- RAG and retrieval strategies
- Multi-agent systems
- Human-in-the-loop
- Tools and external integrations
- Memory and checkpoints
- Error handling and retries
- Caching and performance
- Prompt engineering
- Evaluation metrics
- Deployment (Docker, FastAPI)
- Cloud platforms (AWS/GCP/Azure)
- Monitoring (LangSmith, Grafana)
- Security and best practices
- Cost optimization
- Real-world project experience
- Common challenges and solutions
- Latest trends and updates

19.9 Final Revision Checklist

- ☐ Understand all core concepts
- ☐ Practice coding examples
- ☐ Revise system design diagrams
- ☐ Go through common interview questions
- ☐ Be ready to explain your own projects
- ☐ Practice explaining trade-offs and decisions
- ☐ Understand real-world challenges
- ☐ Keep updated with latest developments

19.10 Summary

- LangGraph is a powerful framework for building stateful and multi-agent AI applications.
- Focus on concepts, coding, architecture and real-world scenarios.
- Practice and revision will help you perform well in interviews.

Rahul Yadav